User Manual

DAJ/AR/TN-2017-14924 Low-Complexity Near-Lossless

Document version: 1.1
Date: 27 Nov. 2019

UAB
Universitat Autònoma
de Barcelona

dEIC
Departament d'Enginyeria
de la Informació
i de les Comunicacions

GICi
Group on Interactive
Coding of Images

Document Information

Document identifier: docs/user_manual.md
Title: User Manual
Number of pages: 20
Date: 27 Nov. 2019
Author(s): Miguel Hernández-Cabronero, Ian Blanes, Joan Serra-Sagristà

Change Control

Revision	Date	Changes
1.0	17 July 2018	(initial version)
1.1	27 Nov. 2019	Support for CCSDS-123.0-B-2

Contents

1 INTRODUCTION

This document is the user manual of a set of command-line tools implementing the functionality described in standards CCSDS-121.0-B-3 and CCSDS-123.0-B-2 as part of the project DAJ/AR/TN-2017-14924 «Low-Complexity Near-Lossless Multispectral & Hyperspectral Data Compression» that Universitat Autònoma de Barcelona (UAB) has developed for Centre National d'Études Spatiales (CNES).

2 TOOLS AND WORKFLOW

2.1 Tools

There are eight tools. Three of those are related to CCSDS-121.0-B-3 and five are related to CCSDS-123.0-B-2. Namely, tools `ldc_header_tool`, `ldc_encoder`, and `ldc_decoder` are related to the former standard, while tools `lcnl_header_tool`, `lcnl_supplementary_tables_encoder`, `lcnl_encoder`, `lcnl_decoder`, and `lcnl_supplementary_tables_decoder` are related to the latter.

For the CCSDS-121.0-B-3 standard, the principal tool is `ldc_encoder`, which takes raw data and encodes them into compressed files. The `ldc_decoder` tool performs the opposite function, i.e., it takes compressed files and produces the original encoded data.

Similarly, for the CCSDS-123.0-B-2 standard the principal tool is `lcnl_encoder`, which takes multi- and hyperspectral image data and encodes them into compressed files. The `lcnl_decoder` tool performs the opposite function, i.e., it takes compressed files and produces the original image data, or an approximate reconstruction of the original image data if the encoding process was lossy.

Tools `ldc_header_tool` and `lcnl_header_tool` are used to produce configuration files (headers) that `ldc_encoder` and `lcnl_encoder` may employ to control the respective encoding processes.

The CCSDS-123.0-B-2 standard supports the embedding of supplementary information tables into compressed files. The `lcnl_supplementary_tables_encoder` tool encodes plain text supplementary information tables into binary tables, which can be embedded into configuration files by using the `lcnl_header_tool` tool, and then into compressed files by using the `lcnl_encoder` tool.

The `lcnl_supplementary_tables_decoder` tool decodes supplementary information tables from compressed files.

2.2 Workflow

The expected workflow in relation with the CCSDS-121.0-B-3 standard is as follows:

1. Produce a configuration file (header) by using `ldc_header_tool`.
2. Use previously produced configuration file with the `ldc_encoder` tool to encode raw data into a compressed data file. The configuration file is embedded into the compressed file.
3. Decode the compressed data file into raw data employing the `ldc_decoder` tool. The configuration is not necessary for decoding.

The expected workflow in relation with the CCSDS-123.0-B-2 standard is as follows:

1. Optionally, produce a binary supplementary information tables file with the `lcnl_supplementary_tables_encoder` tool.
2. Produce a configuration file (header) by using `lcnl_header_tool`. Parts of the configuration file may be deemed optional.
3. Use previously produced configuration file with the `lcnl_encoder` tool to encode multi- and hyperspectral image data into a compressed image file. The configuration file is embedded into the compressed file, except for optional configuration file parts, which are not embedded into the compressed file.
4. Decode the compressed image file into image data employing the `lcnl_decoder` tool. The configuration is not necessary for decoding, except for parts of the configuration file previously deemed optional, which need to be made available to the `lcnl_decoder` tool.
5. The `lcnl_supplementary_tables_decoder` can then be used to extract supplementary information tables from compressed data files.

To increase error resilience, among other reasons, it may be necessary to partition a large image into smaller images. Images partitioned along the along-track dimension can be encoded efficiently by preserving the internal state of the

`lcnl_encoder`. When encoding one of the smaller images, the `lcnl_encoder` tool can be instructed to produce an `encoder_state` file. This file will increase the compression efficiency of the `lcnl_encoder` tool when encoding the immediately posterior image resulting from the partitioned large image.

3 COMMAND-LINE TOOLS USAGE

3.1 ldc_header_tool

3.1.1 Name

`ldc_header_tool` - Header manipulation tool for CCSDS-121.0-B-3.

3.1.2 Diagram

```
                      +-----------------+
  default_header --->|                  |---> header
                     | ldc_header_tool  |
       [options] --->|                  |
                     +-----------------+
```

3.1.3 Synopsis

```
ldc_header_tool [-d default_header]
                [-x parameter=value]...
                [-h] [-v]
                header
```

3.1.4 Description

The command-line program `ldc_header_tool` creates a `header` file describing the configuration of a CCSDS-121.0-B-3 encoder.

It reads a `default_header` file and it modifies the represented CCSDS-121.0-B-3 configuration according to various command-line options. It writes the result to an output `header` file.

If no `default_header` is given, an internal default header is employed.

This tool operates by modifying an internal header copy as options are consumed one by one. Thus, setting a parameter with option `-x` overrides any previously value that was set for that parameter. Similarly, loading a `default_header` file, overrides any previous use of the `-x` option.

3.1.5 Options

-d header

Read a default header from the `default_header` file.

-x parameter=value

Set the `option` field to `value`. Recognized `-x` options are:

- `large_b`: output word size in bytes (from 1 to 8).
- `preprocessor_status`: whether prediction and mapping shall take place before entropy coding (0: no, 1: yes).
- `predictor_type`: predictor type (0: bypass, 1: unit delay, 7: application specific). Must be 0 when preprocessor is absent.
- `mapper_type`: mapper type (0: default mapper, 3: application specific). Must be 0 when preprocessor is absent.
- `data_sense`: whether data is signed or not (0: yes, 1: no).
- `n`: input data sample resolution (from 1 to 32).

- `large_j`: block size (8, 16, 32, or 64).
- `restricted_code_options_flag`: use restricted set of code options (0: no, 1: yes). Must be 0 when `n` >= 5.
- `r`: reference sample period (from 1 to 4096).
- `large_n`: number of input samples (from 1 to 2^48).

-v

Tool shows version information.

-h

Tool shows help information.

3.1.6 Data format specifications

Expected (or produced) file formats are as follows.

- `header`: A bitstream containing a header as per §7.2.1 HEADER of CCSDS-121.0-B-3.

3.1.7 Usage example

The following example loads a header from the `header1` file, then modifies it and writes the result into the `header2` file. Modifications are made to the `data_sense` and `n` parameters to signal that the `ldc_encoder` tool should produce compressed files for unsigned 32-bit data.

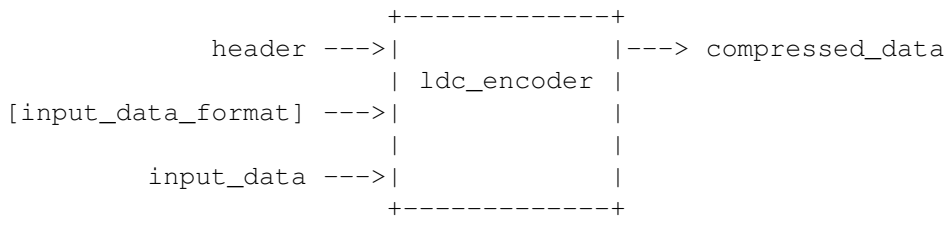
```
ldc_header_tool -d header1 -x data_sense=1 -x n=32 header2
```

3.2 ldc_encoder

3.2.1 Name

ldc_encoder - A CCSDS-121.0-B-3 encoder

3.2.2 Diagram



3.2.3 Synopsis

```
ldc_encoder header input_data_format input_data compressed_data
```

3.2.4 Description

The command-line program `ldc_encoder` produces a `compressed_date` file that is compliant with the CCSDS-121.0-B-3 standard.

The encoder configuration is read from the `header` file, which may have been generated by `ldc_header_tool`.

Data are read from an `input_data` file in the format described in the `input_data_format` command line argument.

The parameter `input_data_format` shall be one of the following data type descriptions: `s8le`, `s8be`, `u8le`, `u8be`, `s16le`, `s16be`, `u16le`, `u16be`, `s32le`, `s32be`, `u32le`, `u32be`.

3.2.5 Options

-v

Tool shows version information.

-h

Tool shows help information.

3.2.6 Data format specifications

Expected (or produced) file formats are as follows.

- `compressed_data`: A file compliant with CCSDS-121.0-B-3.
- `header`: A bitstream containing a header as per §7.2.1 HEADER of CCSDS-121.0-B-3.
- `input_data`: A raw sequence of integers either in unsigned or two's complement format, encoded as specified by `input_data_format` in one of the following 12 data types: `s8le`, `s8be`, `u8le`, `u8be`, `s16le`, `s16be`, `u16le`, `u16be`, `s32le`, `s32be`, `u32le`, `u32be`. A data type is a triplet of the form `{u,s}{8,16,32}{le,be}`, where:
  - the first element denotes whether the data is signed or unsigned,
  - the second element denotes whether the data is stored in the least-significant bits of 8, 16 or 32 bit words, and
  - the third element denotes whether the data is stored in little endian (`le`) or big endian (`be`).

  The number of samples in `input_data` shall be a multiple of the block size, as specified by the `large_j` option of `ldc_header_tool`.

3.2.7 Usage example

The following example encodes the contents of the `data` file, assuming data is stored in the `s16be` format and by using the configuration file `header`. The resulting compressed data is written into the `compressed` file.

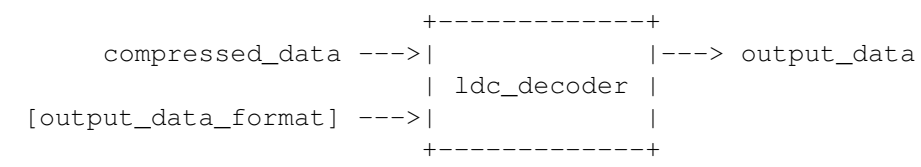
```
ldc_encoder header s16be data compressed
```

3.3 ldc_decoder

3.3.1 Name

`ldc_decoder` - A CCSDS-121.0-B-3 decoder

3.3.2 Diagram



3.3.3 Synopsis

```
ldc_decoder compressed_data output_data_format output_data
```

3.3.4 Description

The command-line program `ldc_decoder` decodes a `compressed_data` file into the `output_data` file. The `compressed_data` file shall be compliant with CCSDS-121.0-B-3.

Data are written to the `output_data` file in the format described in the `output_data_format` command line argument.

The parameter `output_data_format` shall be one of the following data type descriptions: `s8le`, `s8be`, `u8le`, `u8be`, `s16le`, `s16be`, `u16le`, `u16be`, `s32le`, `s32be`, `u32le`, `u32be`.

3.3.5 Options

-v

Tool shows version information.

-h

Tool shows help information.

3.3.6 Data format specifications

Expected (or produced) file formats are as follows.

- `compressed_data`: A file compliant with CCSDS-121.0-B-3.
- `output_data`: A raw sequence of integers either in unsigned or two's complement format, encoded as specified by `output_data_format` in one of the following 12 data types: `s8le`, `s8be`, `u8le`, `u8be`, `s16le`, `s16be`, `u16le`, `u16be`, `s32le`, `s32be`, `u32le`, `u32be`. A data type is a triplet of the form `{u,s}{8,16,32}{le,be}`, where:
  - the first element denotes whether the data is signed or unsigned,
  - the second element denotes whether the data is stored in the least-significant bits of 8, 16 or 32 bit words, and
  - the third element denotes whether the data is stored in little endian (`le`) or big endian (`be`).

3.3.7 Usage example

The following example decodes the contents of the `compressed` file into the `recovered` file. Result is stored in the `s16be` format.

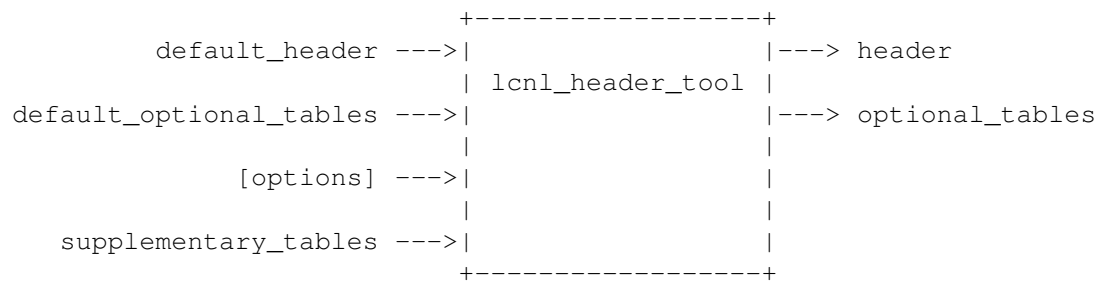
```
ldc_decoder compressed s16be recovered
```

3.4 lcnl_header_tool

3.4.1 Name

`lcnl_header_tool` - Header manipulation tool for CCSDS-123.0-B-2.

3.4.2 Diagram



3.4.3 Synopsis

```
lcnl_header_tool [-d default_header]
                 [-t default_optional_tables]
                 [-s supplementary_tables]
                 [-o optional_tables]
                 [-x parameter=value]...
                 [-h] [-v]
                 header
```

3.4.4 Description

The command-line program `lcnl_header_tool` creates a `header` file describing the configuration of a CCSDS-123.0-B-2 encoder according to various command-line options.

While operating, this tool maintains an internal header copy, which is modified as options are consumed one by one. Once all options are consumed, the result is written to the output `header` file. The initial state of the internal copy is unspecified.

A `default_header` file can be read with option `-d` to override the internal header copy. The values for individual parameters can be set with option `-x`.

The order in which options are given is important, as setting a parameter with option `-x` overrides any previously set value for that parameter. Similarly, loading a `default_header` file, overrides any previous use of the `-x` option.

Header files for CCSDS-123.0-B-2 may not contain all parameter information required by the decoder, under the assumption that these values are already known by the decoder. In practice, this means that a `default_header` may require some optional tables to fully represent the configuration of a CCSDS-123.0-B-2 encoder. Whenever optional parts are not present in a `default_header` file, these shall be previously given in a separate `default_optional_tables` file through option `-t`. Similarly, optional parts not present in the output `header` file, can be stored in the `optional_tables` file through option `-o`.

In addition, the `lcnl_header_tool` will embed into the `header` file any supplementary information table provided either within a `default_header` or through option `-s`. Occurrences of `-d` and `-s` options override the contents of supplementary information tables provided by previous occurrences of either option.

3.4.5 Options

-d default_header

Overrides current internal header with a header read from the `default_header` file. Sets the `default_header` file as the source of supplementary information tables.

-t default_optional_tables

Sets the `default_optional_tables` file to be used when reading a `default_header` file with option `-d`.

-s supplementary_tables

Sets the `supplementary_tables` file as the source of supplementary information tables.

-o optional_tables

When producing the final `header` file, any optional tables not included in the aforementioned file will be included in the `optional_tables` file.

-x option=value

Set the `option` field to `value`. Recognized `-x` options are:

- `user_defined_data`: user defined data field (from 0 to 255).
- `large_n_x`: image size along the x dimension (from 1 to 65536). Should match across-track dimension.
- `large_n_y`: image size along the y dimension (from 1 to 65536). Should match along-track dimension.
- `large_n_z`: image size along the z dimension (from 1 to 65536). Should match spectral dimension. When changing this parameter, vector parameters as described below are extended by repeating their respective last values.
- `sample_type`: image sample type (0: unsigned, 1: signed).
- `large_d`: image bit depth (from 1 to 32).
- `sample_encoding_order`: image order (0: samples are in band-interleaved order, 1: samples are in band-sequential order). Input image must match this order.

- `large_m`: sub-frame interleaving depth (from 1 to large_n_z). Use 1 for band interleaved by line and use large_n_z for band interleaved by pixel.
- `large_b`: output word size (from 1 to 8).
- `entropy_coder_type`: entropy coder (0: sample-adaptive entropy coder, 1: hybrid entropy coder, 2: block-adaptive entropy coder).
- `quantizer_fidelity_control_method`: quantization configuration (0: lossless, 1: absolute error limit only, 2: relative error limit only, 3: both absolute and relative error limits).
- `supplementary_information_table_count`: number of supplementary information tables (from 0 to 15).
- `supplementary_information_table_N_table_type`: data type for table `N` (0: unsigned integer, 1: signed integer, 2: float).
- `supplementary_information_table_N_table_purpose`: purpose of table `N` (0: scale, 1: offset, 2: wavelength, 3: full width at half maximum, 4: defect indicator, 5-9: reserved, 10-15: user-defined).
- `supplementary_information_table_N_table_structure`: structure of table `N` (0: zero-dimensional, 1: one-dimensional, 2: two-dimensional-zx, 3: two-dimensional-yx).
- `supplementary_information_table_N_supplementary_user_defined_data`: user-defined data for table `N` (from 0 to 15).
- `supplementary_information_table_N_large_d_large_i`: bit depth for table `N` of integer type (from 1 to 32).
- `supplementary_information_table_N_large_d_large_f`: significant bit depth for table `N` of float type (from 1 to 23).
- `supplementary_information_table_N_large_d_large_e`: exponent bit depth for table `N` of float type (from 2 to 8).
- `supplementary_information_table_N_beta`: exponent bias for table `N` of float type (unsigned value that must fit in `supplementary_information_table_N_large_d_large_e` bits).
- `sample_representative_flag`: signals inclusion of the sample representative header subpart (0: not included, 1: included).
- `large_p`: number of prediction bands (from 0 to 15).
- `prediction_mode`: prediction mode (0: full, 1: reduced).
- `weight_exponent_offset_flag`: enables non-zero weight exponent offsets (0: disabled, 1: enabled).
- `local_sum_type`: local sum type (0: wide neighbor-oriented, 1: narrow neighbor-oriented, 2: wide column-oriented, 3: narrow column-oriented).
- `large_r`: register size (from 32 to 64).
- `large_omega`: weight component resolution (from 4 to 19).
- `log_two_t_inc`: logarithm in base two of the weight update scaling exponent change interval (from 4 to 11).
- `v_min`: weight update scaling exponent initial parameter (from -6 to 9).
- `v_max`: weight update scaling exponent final parameter (from -6 to 9).
- `weight_exponent_offset_table_flag`: signals inclusion of weight exponent offset table (0: not included, 1: included).
- `weight_initialization_method`: weight initialization method (0: default, 1: custom).
- `weight_initialization_table_flag`: signals inclusion of weight initialization table (0: not included, 1: included).
- `large_q`: custom weight initialization resolution Q (from 3 to 22).

- `large_lambda_vector`: weight initialization table. As a comma-separated vector of values. For each image band, in order of increasing band index `z`, the vector must contain `C_z` consecutive elements (where `C_z` is `P*_z` in reduced prediction mode and `P*_z` + 3 in full prediction mode), where each element corresponds to an element of the weight vector. This parameter depends on the values of `large_n_z`, `large_p` and `prediction_mode`, which must be previously set.
- `zeta_vector`: weight exponent offset table. As a comma-separated vector of values. For each image band, in order of increasing band index `z`, the vector must contain `P*_z` consecutive elements in reduced prediction mode and `P*_z` + 1 consecutive elements in full prediction mode, where each element corresponds to an element in the Weight Exponent Offset Table. This parameter depends on the values of `large_n_z`, `large_p` and `prediction_mode`, which must be previously set.
- `periodic_updating_flag`: enables periodic error limit updating (0: disabled, 1: enabled).
- `u`: error limit update period exponent (from 0 to 9).
- `absolute_error_limit_assignment_method`: absolute error limit assignment method (0: band-independent, 1: band-dependent).
- `large_d_large_a`: absolute error limit bit depth (from 1 to 16).
- `a_vector`: per-band absolute error limit. As a comma-separated vector of `large_d_large_a`-bit unsigned binary integer values. All values must be equal under band-independent absolute error limits. This parameter depends on the value of `large_n_z`, which must be previously set.
- `relative_error_limit_assignment_method`: relative error limit assignment method (0: band-independent, 1: band-dependent).
- `large_d_large_r`: relative error limit bit depth (from 1 to 16).
- `r_vector`: per-band relative error limit. As a comma-separated vector of `large_d_large_r`-bit unsigned binary integer values. All values must be equal under band-independent relative error limits. This parameter depends on the value of `large_n_z`, which must be previously set.
- `large_theta`: sample representative resolution (from 0 to 4). Must not be 0 if, and only if, sample_representative_flag is '1'.
- `band_varying_damping_flag`: damping assignment method (0: band-independent, 1: band-dependent).
- `damping_table_flag`: signals inclusion of damping table (0: not included, 1: included).
- `band_varying_offset_flag`: offset assignment method (0: band-independent, 1: band-dependent).
- `offset_table_flag`: signals inclusion of offset table (0: not included, 1: included).
- `phi_vector`: per-band sample representative damping values. As a comma-separated vector of `large_theta`-bit unsigned binary integer values. All values must be equal when `band_varying_damping_flag` is 0. This parameter depends on the value of `large_n_z`, which must be previously set.
- `psi_vector`: per-band sample representative offset values. As a comma-separated vector of `large_theta`-bit unsigned binary integer values. All values must be equal when `band_varying_offset_flag` is 0. This parameter depends on the value of `large_n_z`, which must be previously set.
- `large_u_max`: unary length limit (from 8 to 32).
- `gamma_star`: rescaling counter size (from 4 to 11).
- `gamma_0`: initial count exponent (from 1 to 8).
- `band_varying_k_flag`: accumulator initialization method (0: band-independent, 1: band-dependent).
- `accumulator_initialization_table_flag`: signals inclusion of accumulator initialization table (0: not included, 1: included).
- `k_prime_prime_vector`: per-band accumulator initialization values. As a comma-separated vector of 4-bit unsigned binary integer values. All values must be equal when `band_varying_k_flag` is 0. This parameter depends on the value of `large_n_z`, which must be previously set.
- `large_j`: block size (8, 16, 32, or 64).

- `restricted_code_options_flag`: use restricted set of codes (0: disable, 1: enable). Requires `large_d` <= 4.
- `r`: reference sample interval (from 1 to 4096).

Parameters with names ending in `_vector` receive a comma-separated vector of integer values (without separating blanks). For these parameters, the last element of the vector is repeated as many times as needed, when less than the required number of elements is provided. For example, `-x a_vector=2` sets an absolute error limit of 2 for all bands.

-v

Tool shows version information.

-h

Tool shows help information.

3.4.6 Data format specifications

Expected (or produced) file formats are as follows.

- `header`: A bitstream containing a header as per §5.3 HEADER of CCSDS-123.0-B-2.
- `optional_tables`: The concatenation of the bitstreams for the equivalent associated tables in §5.3 HEADER of CCSDS-123.0-B-2, which are not present in an associated `header` file.
- `supplementary_tables`: A bitstream containing the concatenation of the sequences of table elements for all tables, as described in 5.3.2.2.1.1 b) or 5.3.2.2.1.2 d) of CCSDS-123.0-B-2.

3.4.7 Usage example

The following example loads a header from the `header1` file, then modifies it and writes the result into the `header2` file. Modifications are made to the `large_n_x`, `large_n_y`, `large_n_z`, `sample_type`, and `large_d` parameters.

```
lcnl_header_tool -d header1 -x large_n_x=32 -x large_n_y=40 \\
    -x large_n_z=16 -x sample_type=0 -x large_d=16 header2
```

The following example takes a header file and makes the weight exponent offset table optional.

```
lcnl_header_tool -t header1_optional_parts -d header1
    -x weight_exponent_offset_table_flag=0 \\
    -o header2_optional_parts header2
```

The following example enables sample representatives, enables band-varying damping, and sets the damping parameter (phi) to zero for all bands, except for the second one.

```
lcnl_header_tool
    -x large_theta=2
    -x band_varying_damping_flag=1
    -x phi_vector=0,1,0
    header
```

Note how the last `0` in `phi_vector` is automatically repeated as needed.

The following example extracts the encoder configuration from a `compressed_image` file and stores it in the `header` file.

```
lcnl_header_tool -d compressed_image header
```

The following example creates a new header file with a supplementary information table, provided that the contents of the supplementary information table are available in a binary file.

```
lcnl_header_tool -x supplementary_information_table_count=1 \\
   -x supplementary_information_table_0_table_type=1 \\
   -x supplementary_information_table_0_table_structure=0 \\
   -x supplementary_information_table_0_large_d_large_i=16  \\
   -s sit.bin
   header
```

The following example extends the previous one by obtaining the information from a text file with a CSV-like format. First a header with the placeholder for the table information is created by loading an empty table from /dev/zero. A plain text file is then parsed into a binary supplementary information table. Finally, the binary file is then embedded into the header.

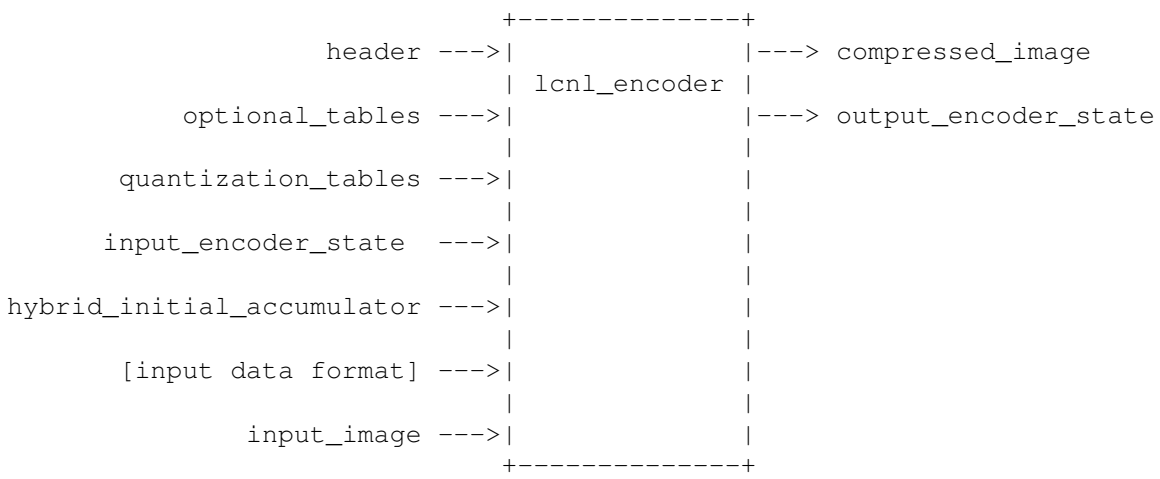
```
lcnl_header_tool -x supplementary_information_table_count=1 \\
   -x supplementary_information_table_0_table_type=1 \\
   -x supplementary_information_table_0_table_structure=0 \\
   -x supplementary_information_table_0_large_d_large_i=16  \\
   -s /dev/zero
   header_tmp

lcnl_supplementary_tables_encoder header_tmp sit.bin table_0.txt

lcnl_header_tool -d header_tmp -s sit.bin header
```

3.5 lcnl_encoder

3.5.1 Name

lcnl_encoder - A CCSDS-123.0-B-2 encoder.

3.5.2 Diagram



3.5.3 Synopsis

```
lcnl_encoder [-o optional_tables]
             [-q quantization_tables]
             [-a hybrid_initial_accumulator]
             [-i input_encoder_state]
             [-e output_encoder_state]
             header input_data_format input_image compressed_image
```

3.5.4 Description

The command line program `lcnl_encoder` produces CCSDS-123.0-B-2 compliant files.

It reads a `header` file and, optionally, an `optional_tables` file describing the configuration of the CCSDS-123.0-B-2 encoder to be employed.

It reads data from an `input_image` file in the format described in the `input_data_format` command line argument, and compresses the data according to CCSDS-123.0-B-2 into a `compressed_image` file.

The parameter `input_data_format` shall be one of the following data type descriptions: `s8le`, `s8be`, `u8le`, `u8be`, `s16le`, `s16be`, `u16le`, `u16be`, `s32le`, `s32be`, `u32le`, `u32be`.

Any quantization adjustments not present in the `header` file (see §5.3.3.3.2.2.1 and §5.3.3.3.3.2.1 of CCSDS-123.0-B-2) are read from an optional `quantization_tables` file.

Optionally, an `output_encoder_state` file is produced representing the terminating state of a CCSDS-123.0-B-2 encoder, so that a different instance of an `lcnl_encoder` may adjust a `header` file to incorporate the settings learned while producing `compressed_image` by using that file as an `input_encoder_state` file.

3.5.5 Options

-o optional_tables

Read any missing header tables in the `header` file from the `optional_tables` file.

-i encoder_state

Read an `input_encoder_state` file representing the terminating state of a CCSDS-123.0-B-2 encoder.

-q quantization_tables

Read any quantization adjustments not present in the `header` file from the `quantization_tables` file.

-e encoder_state

Produce an `output_encoder_state` file representing the terminating state of a CCSDS-123.0-B-2 encoder.

-a hybrid_initial_accumulator

Read initial accumulator values for the hybrid encoder from the `hybrid_initial_accumulator` file.

-v

Tool shows version information.

-h

Tool shows help information.

3.5.6 Data format specifications

Expected (or produced) file formats are as follows.

- `compressed_image`: A bitstream compliant with CCSDS-123.0-B-2.
- `encoder_state`: A, possibly non-portable, opaque binary file. No guarantees are provided regarding this file other than that an `encoder_state` produced by the `lcnl_encoder` tool will be correctly read by the `lcnl_header_tool` utility when both tools have been compiled for the same target architectures employing the same toolchain and build options.
- `header`: A bitstream containing a header as per §5.3 HEADER of CCSDS-123.0-B-2.
- `hybrid_initial_accumulator` A raw sequence of unsigned integers of `large_d` + `gamma_0` bits.
- `input_image`: A raw sequence of integers either in unsigned or two's complement format, encoded as one of the following 12 data types: `s8le`, `s8be`, `u8le`, `u8be`, `s16le`, `s16be`, `u16le`, `u16be`, `s32le`, `s32be`, `u32le`, `u32be`. A data type is a triplet of the form `{u,s}{8,16,32}{le,be}`, where:
  - the first element denotes whether the data is signed or unsigned,
  - the second element denotes whether the data is stored in the least-significant bits of 8, 16 or 32 bit words, and
  - the third element denotes whether the data is stored in little endian (`le`) or big endian (`be`).

Data needs to be in the order described in §5.4.2 INPUT ORDER of CCSDS-123.0-B-2 as specified in the `header` file. E.g., data needs to be in band-sequential order when `Sample Encoding Order` is set to 1.

- `optional_tables`: The concatenation of the bitstreams for the equivalent associated tables in §5.3 HEADER of CCSDS-123.0-B-2, which are not present in an associated `header` file.
- `quantization_tables`: A sequence of 16-bit integers in big-endian unsigned format, in the same order specified in §5.4.2 INPUT ORDER of CCSDS-123.0-B-2.

3.5.7 Usage example

The following example encodes an `input_image` file, stored as 16-bit big-endian unsigned integers, into a `compressed_image` file. Configuration is read from the `header` file and from the `optional_tables` file.

```
lcnl_encoder -o optional_tables header u16be input_image compressed_image
```

The following example encodes an `input_image` file, stored as 32-bit little-endian signed integers, into a `compressed_image` file. Configuration is read from the `header` file. Quantization tables and the hybrid initial accumulator values are read from files `quantization_tables` and `hybrid_initial_accumulator`, respectively.

```
lcnl_encoder -q quantization_tables -a hybrid_initial_accumulator \\
    header s32le input_image compressed_image
```

In the following example a large image has been partitioned into three parts of identical dimensions along the along-track dimension. Parts are named `input_image_top`, `input_image_middle` and `input_image_bottom`. Images are encoded successively, with the last two parts reusing the encoder state as it is left by the previous encoding process.

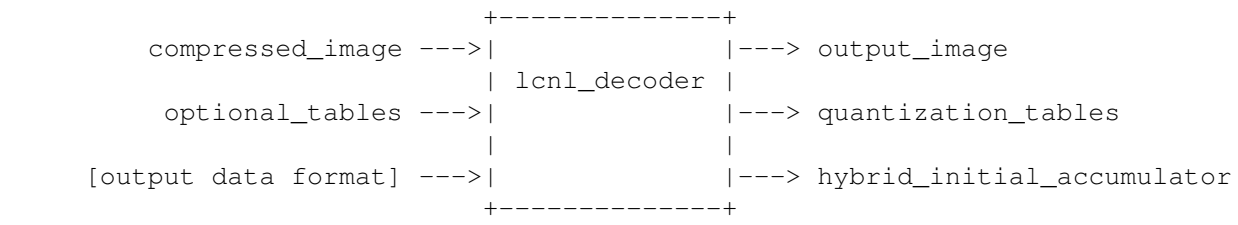
```
lcnl_encoder -e encoder_state_top \\
    header s16be input_image_top compressed_image_top

lcnl_encoder -i encoder_state_top -e encoder_state_middle \\
    header s16be input_image_middle compressed_image_middle

lcnl_encoder -i encoder_state_middle \\
    header s16be input_image_bottom compressed_image_bottom
```

3.6 lcnl_decoder

3.6.1 Name

lcnl_decoder - A CCSDS-123.0-B-2 decoder.

3.6.2 Diagram



3.6.3 Synopsis

```
lcnl_decoder [-o optional_tables]
             [-q quantization_tables]
             [-a hybrid_initial_accumulator]
             compressed_image output_data_format output_image
```

3.6.4 Description

The command-line program `lcnl_decoder` decodes a `compressed_image` file into the `output_image` file. The `compressed_data` file shall be compliant with CCSDS-123.0-B-2.

Data are written to the `output_image` file in the format described in the `output_data_format` command line argument.

The parameter `output_data_format` shall be one of the following data type descriptions: `s8le`, `s8be`, `u8le`, `u8be`, `s16le`, `s16be`, `u16le`, `u16be`, `s32le`, `s32be`, `u32le`, `u32be`.

3.6.5 Options

-o optional_tables

Read any header tables missing in the `compressed_image` file from the `optional_tables` file.

-q quantization_tables

Write any quantization adjustments not present in the header part of the `compressed_image` file. This file can be employed by `lcnl_encoder` to reproduce an identical copy of `compressed_image`.

-a hybrid_initial_accumulator

Write initial accumulator values obtained by the hybrid decoder to the `hybrid_initial_accumulator` file. This file can be employed by `lcnl_encoder` to reproduce an identical copy of `compressed_image`.

-v

Tool shows version information.

-h

Tool shows help information.

3.6.6 Data format specifications

Expected (or produced) file formats are as follows.

- `compressed_image`: A bitstream compliant with CCSDS-123.0-B-2.
- `hybrid_initial_accumulator` A raw sequence of unsigned integers of `large_d` + `gamma_0` bits.
- `output_image`: A raw sequence of integers either in unsigned or two's complement format, encoded as one of the following 12 data types: `s8le`, `s8be`, `u8le`, `u8be`, `s16le`, `s16be`, `u16le`, `u16be`, `s32le`, `s32be`, `u32le`, `u32be`. A data type is a triplet of the form `{u,s}{8,16,32}{le,be}`, where:
  - the first element denotes whether the data is signed or unsigned,
  - the second element denotes whether the data is stored in the least-significant bits of 8, 16 or 32 bit words, and
  - the third element denotes whether the data is stored in little endian (`le`) or big endian (`be`).

  Data is in the order described in §5.4.2 INPUT ORDER of CCSDS-123.0-B-2 as specified in the `compressed_image` file. E.g., data is in band-sequential order when `Sample Encoding Order` is set to 1.
- `optional_tables`: The concatenation of the bitstreams for the equivalent associated tables in §5.3 HEADER of CCSDS-123.0-B-2, which are not present in an associated `header` file.
- `quantization_tables`: A sequence of 16-bit integers in big-endian unsigned format, in the same order specified in §5.4.2 INPUT ORDER of CCSDS-123.0-B-2.

3.6.7 Usage example

The following example decodes a `compressed_image` file into an `output_image` file, stored as 16-bit big-endian unsigned integers. Optional configuration is read from the `optional_tables` file.

```
lcnl_decoder -o optional_tables compressed_image u16be output_image
```

The following example decodes a `compressed_image` file (without optional tables) into an `output_image` file, and re-encodes again the `output_image` into a `compressed_image2` file. Both compressed image files are identical.

```
lcnl_decoder -a hybrid_initial_accumulator -q quantization_tables \\
    compressed_image u16be output_image

lcnl_header_tool -d compressed_image header

lcnl_encoder -q quantization_tables -a hybrid_initial_accumulator \\
    header u16be output_image compressed_image2
```

3.7 lcnl_supplementary_tables_encoder

3.7.1 Name

lcnl_supplementary_tables_encoder - Supplementary tables creation for CCSDS-123.0-B-2.

3.7.2 Diagram

3.7.3 Synopsis

```
lcnl_supplementary_tables_encoder [-o optional_tables]
                                  header
                                  binary_tables
                                  plain_table_0...
```

3.7.4 Description

The `lcnl_supplementary_tables_encoder` tool encodes up to 15 plain-text supplementary information tables into a `binary_tables` file.

Tables are read from `plain_table_N` files according to the configuration described by `header` and `optional_tables`.

3.7.5 Options

-v

Tool shows version information.

-h

Tool shows help information.

3.7.6 Data format specifications

Expected (or produced) file formats are as follows.

- `header`: A bitstream containing a header as per §5.3 HEADER of CCSDS-123.0-B-2.
- `optional_tables`: The concatenation of the bitstreams for the equivalent associated tables in §5.3 HEADER of CCSDS-123.0-B-2, which are not present in an associated `header` file.
- `plain_table_N`: A CSV-like file containing comma-delimited plain-text values. Values are expected in the orders given in 5.3.2.2.1.1 b) and 5.3.2.2.1.2 d) of CCSDS-123.0-B-2.
- `binary_tables`: A bitstream containing the concatenation of the sequences of table elements for all tables, as described in 5.3.2.2.1.1 b) or 5.3.2.2.1.2 d) of CCSDS-123.0-B-2.

3.7.7 Usage example

The following example encodes plain-text table files `plain_table_0` and `plain_table_1` into a `binary_tables` file, according to configuration `header` file.

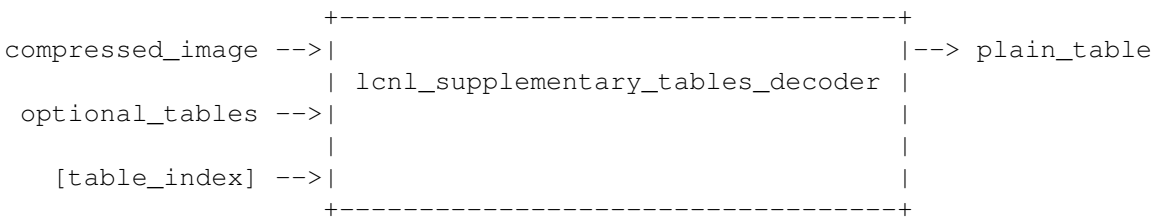
```
lcnl_supplementary_tables_encoder header binary_tables plain_table_0 plain_table_1
```

3.8 lcnl_supplementary_tables_decoder

3.8.1 Name

lcnl_supplementary_tables_encoder - Supplementary tables creation for CCSDS-123.0-B-2.

3.8.2 Diagram



3.8.3 Synopsis

```
lcnl_supplementary_tables_decoder [-o optional_tables]
                                  compressed_image
                                  table_index
                                  plain_table
```

3.8.4 Description

The `lcnl_supplementary_tables_decoder` tool decodes a single plain-text supplementary information table from `compressed_image` into a `binary_tables` file, as specified by `table_index`.

Argument `table_index` must be an integer from 0 to 14.

3.8.5 Options

-v

Tool shows version information.

-h

Tool shows help information.

3.8.6 Data format specifications

Expected (or produced) file formats are as follows.

- `compressed_image`: A bitstream compliant with CCSDS-123.0-B-2.
- `optional_tables`: The concatenation of the bitstreams for the equivalent associated tables in §5.3 HEADER of CCSDS-123.0-B-2, which are not present in an associated `compressed_image` file.
- `plain_table`: A CSV-like file containing comma-delimited plain-text values. Values are produced in the orders given in 5.3.2.2.1.1 b) and 5.3.2.2.1.2 d) of CCSDS-123.0-B-2.

3.8.7 Usage example

The following example decodes the first two supplementary information tables from `compressed_image` into files `plain_table_0` and `plain_table_1`.

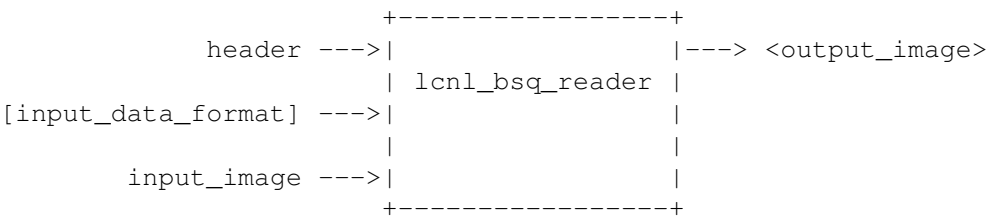
```
lcnl_supplementary_tables_decoder compressed_image 0 plain_table_0
lcnl_supplementary_tables_decoder compressed_image 1 plain_table_1
```

3.9 lcnl_bsq_reader

3.9.1 Name

lcnl_bsq_reader - BSQ-to-any image reordering tool.

3.9.2 Diagram



3.9.3 Synopsis

```
lcnl_bsq_reader header input_data_format input_image
```

3.9.4 Description

The `lcnl_bsq_reader` tool reads an `input_image` file and produces an `output_image` in the order specified in `header`.

The `input_image` samples must be stored in band-sequential order and in the data type specified by `input_data_format`. The `output_image` is written to the standard output.

The parameter `input_data_format` shall be one of the following data type descriptions: `s8le`, `s8be`, `u8le`, `u8be`, `s16le`, `s16be`, `u16le`, `u16be`, `s32le`, `s32be`, `u32le`, `u32be`.

No data type conversion is performed by this tool.

This tool can be employed, together with `lcnl_encoder`, to encode an image in band-sequential order without a temporary reordered image.

3.9.5 Options

-v

Tool shows version information.

-h

Tool shows help information.

3.9.6 Data format specifications

Expected (or produced) file formats are as follows.

- `header`: A bitstream containing a header as per §5.3 HEADER of CCSDS-123.0-B-2.
- `input_image`: A raw sequence of integers either in unsigned or two's complement format, encoded as one of the following 12 data types: `s8le`, `s8be`, `u8le`, `u8be`, `s16le`, `s16be`, `u16le`, `u16be`, `s32le`, `s32be`, `u32le`, `u32be`. A data type is a triplet of the form `{u,s}{8,16,32}{le,be}`, where:
  - the first element denotes whether the data is signed or unsigned,
  - the second element denotes whether the data is stored in the least-significant bits of 8, 16 or 32 bit words, and
  - the third element denotes whether the data is stored in little endian (`le`) or big endian (`be`).

  Data needs to be in the order described in §5.4.2.3 Band-Sequential Order of CCSDS-123.0-B-2.
- `output_image`: A raw sequence of integers either in unsigned or two's complement format, encoded as one of the following 12 data types: `s8le`, `s8be`, `u8le`, `u8be`, `s16le`, `s16be`, `u16le`, `u16be`, `s32le`, `s32be`, `u32le`, `u32be`.

  Data is in the order described in §5.4.2 INPUT ORDER of CCSDS-123.0-B-2 as specified in the `header` file. E.g., data is in band-sequential order when `Sample Encoding Order` is set to 1.

3.9.7 Usage example

The following example demonstrates how to employ this tool to encode a band-sequential image without temporary files.

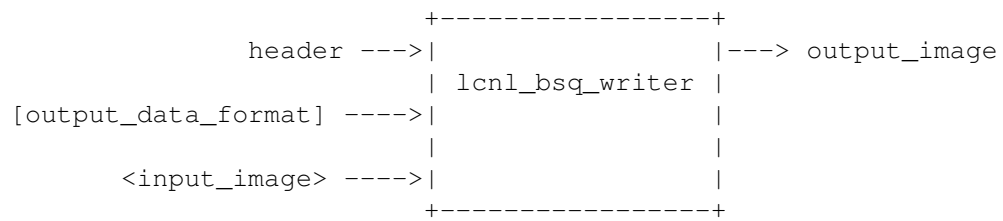
```
lcnl_bsq_reader header s16be bsq_image | \\
    lcnl_encoder header s16be /dev/stdin compressed_image
```

3.10 lcnl_bsq_writer

3.10.1 Name

lcnl_bsq_writer - Any-to-BSQ image reordering tool.

3.10.2 Diagram



3.10.3 Synopsis

```
lcnl_bsq_writer header output_data_format output_image
```

3.10.4 Description

The `lcnl_bsq_writer` tool reads an `input_image` file in the order specified in `header` and produces an `output_image`.

The `input_image` is read from the standard input. The `output_image` samples are stored in band-sequential order and in the data type specified by `input_data_format`.

The parameter `output_data_format` shall be one of the following data type descriptions: `s8le`, `s8be`, `u8le`, `u8be`, `s16le`, `s16be`, `u16le`, `u16be`, `s32le`, `s32be`, `u32le`, `u32be`.

No data type conversion is performed by this tool.

This tool can be employed, together with `lcnl_decoder`, to decode an image in band-sequential order without a temporary reordered image.

3.10.5 Options

-v

Tool shows version information.

-h

Tool shows help information.

3.10.6 Data format specifications

Expected (or produced) file formats are as follows.

- `header`: A bitstream containing a header as per §5.3 HEADER of CCSDS-123.0-B-2.
- `input_image`: A raw sequence of integers either in unsigned or two's complement format, encoded as one of the following 12 data types: `s8le`, `s8be`, `u8le`, `u8be`, `s16le`, `s16be`, `u16le`, `u16be`, `s32le`, `s32be`, `u32le`, `u32be`. A data type is a triplet of the form `{u,s}{8,16,32}{le,be}`, where:
  - the first element denotes whether the data is signed or unsigned,
  - the second element denotes whether the data is stored in the least-significant bits of 8, 16 or 32 bit words, and
  - the third element denotes whether the data is stored in little endian (`le`) or big endian (`be`).

  Data needs to be in the order described in §5.4.2 INPUT ORDER of CCSDS-123.0-B-2 as specified in the `header` file. E.g., data is in band-sequential order when `Sample Encoding Order` is set to 1.
- `output_image`: A raw sequence of integers either in unsigned or two's complement format, encoded as one of the following 12 data types: `s8le`, `s8be`, `u8le`, `u8be`, `s16le`, `s16be`, `u16le`, `u16be`, `s32le`, `s32be`, `u32le`, `u32be`.

  Data is in the order described in §5.4.2.3 Band-Sequential Order of CCSDS-123.0-B-2.

3.10.7 Usage example

The following example demonstrates how to employ this tool to decode a compressed file into a band-sequential image without temporary files.

```
lcnl_decoder -o optional_tables compressed_image u16be /dev/stdout | \\
    lcnl_bsq_writer compressed_image s16be bsq_image
```

4 APPLICABLE DOCUMENTS

[CCSDS-121.0-B-3]: Lossless Data Compression. Blue Book. Issue 3. 2018. (`https://public.ccsds.org/Publications/BlueBooks.aspx`).

[CCSDS-123.0-B-2]: Lossless Multispectral & Hyperspectral Image Compression. Blue Book. Issue 2. 2018. (`https://public.ccsds.org/Publications/BlueBooks.aspx`).